08. How the System Works: From Request to Result

Stage 1: Ingestion & Features

- **Capture request:** Read the user's goal.
- **Clean text:** Remove noise and missing values.
- **Extract features:** Convert request into 10k TF-IDF signals.

Stage 2: Intelligence & DAG

- **Classify intent:** 12 task intents (99.6% accuracy).
- **Choose agents & models:** Match sub-tasks to specialists.
- **Build the DAG:** Structure tasks by dependencies & safety.

Stage 3: Execution & Storage

- **Execute:** Run parallel tasks ($K \leq 5$) with MCP tools.
- **Stream updates:** Live SSE feedback and console logs.
- **Check & save:** ORPO verification and PrismDB storage.

Computer Science Walkthrough: Building & Executing the Task Graph $\mathcal{G}(V, E)$

1. User Goal Prompt → **2. Split into Vertices V** → **3. Connect Edges E** → **4. Topological Run**

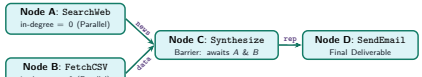
"Analyze news+CSV, email summary" | Modular async tasks A, B, C, D | Map output return $\rightarrow$ input args | Run in-degree $= 0 \implies$ join barrier

Computer Science Explanation (By Concrete Example):

- **Vertices (V) = Sub-Task Functions (Async Call Units):**
  Node A: `news = await SearchWeb("Tesla news")` *[Researcher]*
  Node B: `data = await FetchCSV("metrics.csv")` *[Data Agent]*
  Node C: `rep = await Synthesize(news, data)` *[Writer Agent]*
  Node D: `await SendEmail(to="lead", body=rep)` *[Email Tool]*
- **Edges (E) = Data Pipes & Synchronization Barriers:**
  Edge (A, C) pipes return value `news`; edge (B, C) pipes data.
  Barrier: task C blocks until both promises A & B resolve.
- **Connection Basis (Input $\leftarrow$ Output Matching):**
  Function C links to A & B because C's parameters require their outputs. A and B share no edge because neither depends on the other's data.
- **Execution Schedule (Topological Concurrency):**
  Tasks with in-degree $= 0$ (A, B) execute **concurrently in parallel**.

When A & B complete, C's in-degree drops to $0 \implies C$ runs $\implies D$ runs.

Visual Data-Flow Graph & Execution Order



Parallel Thread 1: Worker 1 executes Node A $\rightarrow$ returns `news`
Parallel Thread 2: Worker 2 executes Node B $\rightarrow$ returns `data`
Join Barrier: Node C consumes `news` + `data` $\rightarrow$ outputs `rep`

Serial Finish: Node D consumes `rep` $\rightarrow$ sends email